

PCIM-SAT: A 55nm Probabilistic K-SAT Solver with p-Bit-Based Parallel-Variable Update on a Mixed-Signal Compute-in-Memory Architecture

Publisher: **IEEE**

Cite This

<< Results

Tinish Bhattacharya ; George Higgins Hutchinson ; Dongseok Kwon ; Dmitri Strukov



1
Cites in
Paper

390
Full
Text Views

Abstract

Authors

Figures

References

Citations

Keywords

Metrics

More Like This



Down
PDF

Abstract:

We present a 55 nm mixed-signal K-SAT solver with parallel-variable update algorithm for improved convergence and a compute-in-memory fabric that maps arbitrary-order K-S... [Show More](#)

Metadata

Contents

Boolean Satisfiability (K -SAT, $K \geq 3$) is an NP-complete problem that arises across nearly all domains of science and engineering like cryptography, computational biology, AI planning and EDA [1]. A K-SAT instance with N variables and M clauses is expressed as a conjunction (AND) of clauses, each being a disjunction (OR) of up to K literals (Boolean variables or their complements, Fig. 10.10.1, top). Solving SAT, i.e., finding satisfying assignment for all clauses, is computationally expensive on any hardware, which has motivated efforts towards designing SAT-solving hardware accelerators [2]–[7]. Complete K-SAT solvers based on the DPLL algorithm [2] can prove unsatisfiability of instances, but their decision-tree search scales poorly with problem size compared to other algorithms. A fully digital WalkSAT-based solver [3] exploited SRAM-based In-Memory Computing (IMC) for parallel clause evaluation but computed break-values (B_i , Fig. 10.10.1, top) sequentially, thereby limiting its performance. A mixed-signal IMC solver [4], [5] improved parallelism by computing clause and break-values in one shot using SRAM-based IMC, but updated only one variable per iteration, limiting navigational efficiency. Solvers updating multiple variables in parallel using chromatic block updates [6] or continuous-time (CT) dynamics [7], [8] have demonstrated fast convergence but are typically restricted to 3-SAT problems. Solving higher-order ($K > 3$)-SAT problems on such hardware [6]–[8], as well as other accelerators that are limited to second-order interactions [9], [10], requires introducing additional variables and clauses, which inflates the search space exponentially and degrades performance [11].

This paper introduces a probabilistic SAT-solving algorithm, that operates directly in the high-order K-SAT space and leverages the bipartite clause-variable relationship to update multiple variables per Monte Carlo step with high navigational efficiency. It modifies the sigmoidal probability function $\sigma(\cdot)$ of Gibbs sampling-based solvers by scaling it with a Heaviside step function $\delta(\cdot)$ of the variable's make-value M_i . This restricts updates to *promising* variables in unsatisfied clauses. Variables accepted by this function are updated independently in parallel, while a rejection-free mechanism ensures progress by updating all *promising* variables with non-zero M_i , if no candidate is accepted. This heuristic outperforms state-of-the-art (SOTA) algorithms such as WalkSAT and the Digital Annealer (when simulated on a third-order Ising formulation of 3-SAT), with performance gains increasing with problem size (Fig. 10.10.1, left).



This work advances the field of SAT-solving hardware by making four key contributions (Fig. 10.10.1): (1) a mixed-signal in-memory computing architecture that is order-invariant and implements a novel SAT solver capable of updating multiple variables in each time step without prior preprocessing, (2) A custom 11T SRAM crossbar array that allows computation of both make and break-values of all variables (effectively the gradients) in a single cycle, (3) dynamic comparator-based Probabilistic-bit (p-bit) circuits on the variable side, enabling massively parallel sampling informed by the analog gradients computed in the crossbar, eliminating the need for ADCs or analog-domain reductions, (4) multi-threaded pipeline that exploits the decoupled orthogonal accesses to the crossbar array to launch three independent and parallel solvers, improving convergence by $1.7\times$ at similar energy. Our prototype fabricated in 55 nm, outperforms SOTA solvers, including continuous-time implementations in more advanced 28 nm [8] and discrete-time solvers [4] by $4 - 13\times$ on uniform random 3-SAT instances having up to 50 variables and on practically relevant KSAT instances by $2 - 14\times$.

The algorithm implementation (Fig. 10.10.2, right) involves the following circuit blocks (Fig. 10.10.2, left): a 256×128 bidirectional 11T-SRAM array; clause evaluators; variable/clause registers and drivers; satisfiability (SAT) evaluators; make-value filter; p-bits, a 128b Pseudo Random Number Generator (PRNG); and a 128b wide 4 -stage RNG buffer chain. In the first step, the K-SAT problem is loaded into the memory array such that each row corresponds to a clause and each column to a literal. Depending on whether x_i or $\sim x_i$ is present in clause C_j , $\text{SRAM}_{j,2i}$ or $\text{SRAM}_{j,2i+1}$ is set to 1 respectively, otherwise both are 0. In the second step, all variables are initialized to random binary values, using the PRNG. Subsequently the solver attempts to find the solution by iteratively executing the following three functions in sequence: (i) Forward Pass (FP), where literal-values are applied to the crossbar and all clauses are evaluated in parallel; (ii) Backward Pass (BP), where clause signals are applied to the crossbar array to compute the make and break values of all variables in parallel. The make-value filters detect the variables with $M_i > 0$, whereas the p-bits for each variable output a 1 with a probability proportional to the gain of the variable ($= M_i - B_i$). Simultaneously, the SAT evaluator checks if problem is satisfied, (iii) Variable Update (VU), which checks if there are *promising* variables with $M_i > 0$ and accepted by p-bit, in which case all such variables are flipped, otherwise all with $M_i > 0$ are flipped. Existence of at least one variable with $M_i > 0$ is guaranteed at this stage, since no variable with $M_i > 0$ implies the solution has been reached.

On the schematic level (Fig. 10.10.3), each coupling cell in a row consists of a 6T SRAM, 2T forward switch and 3T backward switch. During the forward pass, drivers apply variable-proportional voltages to the RWLFs, while at the same time pull-up pFETs connected to the RBLFs on the clause side are turned on. The forward switches that are ON, represent literals in respective clauses that have been set to 1 (TRUE). As a result, the voltage divider formed by the pFET and the nFETs in the forward switches generates a voltage on RBLF j inversely proportional to the number of true literals in clause C_j . StrongARM latch-based voltage sense amplifiers (VSAs), referenced to off-chip analog reference voltages ref_0 and ref_1 , are used to detect whether a clause has zero ($S_j = 1$) or at most one ($O_j = 1$) true literal. Additional logic derives signal Z, indicating clauses with exactly one true literal. The SAT evaluator uses tree of OR gates to check if all clauses are satisfied. In the Backward Pass, clause signals S_j and Z_j are applied to RWLB S_j and RWLBZ j , respectively. The make-value of variable x_i is evaluated by accumulating contribution from clauses with $S_j = 1$ and that contain its zero-valued literal, on PXM_i . The break-value of variable x_i is computed by accumulating contribution from clauses with $Z_j = 1$ and that contain its one-valued literal on PXB_i [12]. The peripheral pull-up pFETs and MUXes ensure for each variable that a voltage-divider on only the RBLBS (RBLBZ) line of its zero- (one-) valued literal is activated and its output is connected to PXM_i (PXB_i). VSAs referenced to ref_2 are used to detect $M_i > 0$. Probabilistic bits are implemented using a StrongARM latch with three differential pairs: (i) T_1 and T_3 , (ii) T_2 and T_4 , (iii) $T_{5:12}$ and $T_{13:20}$. While both (i) and (ii) discharge nodes P and Q during evaluation phase ($\text{PEN} = 1$), with differential current proportional to $\text{PXB}_i - \text{PXM}_i$, the latter along with its p-type input common-source amplifiers extends support for rail-to-rail input range. Each branch in (iii) is a 4b binary-weighted current switching DAC and the pair contributes a differential current proportional to $\text{ref}_p^* (R_p - R_n)$, where R_p, R_n are the uniform random numbers corresponding to binary bits $\text{rp}_{3:0}$ and $\text{rn}_{3:0}$. Ignoring offset voltage, transconductance variations in the different pairs, the output of the p-bit is one if $(\text{PXB}_i - \text{PXM}_i) > \text{ref}_p \times R_0$, where $R_0 = R_n - R_p$ is also a uniform random number. The differential voltage $(\text{PXB}_i - \text{PXM}_i)$ is a saturating nonlinear function of the variable's gain ($\Delta G_i = M_i - B_i$), leading to an approximately sigmoidal dependence of the p-bit's output probability and the gain. Each p-bit receives 8 random bits from the RNG buffer chain with each of the four buffer stages supplying random bits for a total of $16p$ -bits.

The experimental results for probabilistic-bits (Fig. 10.10.4, left) reveal a relatively tight spread in the characteristics across all p-bits, arising primarily from the offset voltage variations in the circuit's differential pairs. (The results also indicate that the slope of the pbit function, i.e., the solver's temperature parameter T , can be dynamically adjusted by tuning the reference bias (refp), a feature that can be exploited in more advanced algorithms with annealing). While prior SAT accelerators [4] have been restricted by structural hazards in the IMC fabric to single-thread operation, our circuit can interlace three independent solver threads for the same SAT problem in its three-stage pipeline (Fig. 10.10.4, right). While the problem graph mapped to the SRAM array is reused, three sets of variable and clause registers are required for each thread to independently maintain its variable and clause statuses, respectively.

The solver's measured results on common SAT benchmarks show that multi-thread parallelism improves convergence by $1.7\times$, while maintaining similar energy consumption (Fig. 10.10.5). Independent threads parallelly explore different regions of the energy landscape, thereby increasing the ensemble's probability of reaching a solution early. Our chip's performance scaling is compared with several prior solvers not requiring preprocessing. Compared to prior SOTA discrete-time incomplete solvers [4], our solver is 8-to-13 $\times$ faster and 2-to-5 $\times$ more energy efficient on uniform random 3-SAT problems, whereas it is 2-to-14 $\times$ faster on practically relevant higher-order K-SAT problems. Compared to a continuous-time solver in a more advanced 28 nm node [8] our solver is 4-to-5.6 $\times$ faster with comparable energy efficiency. Compared to a purely digital DPLL-based complete solver [2] also in 28 nm, our chip achieves a 3 $\times$ speedup. Modeling in [2] further suggests a 84 $\times$ speedup on larger problems.

In summary, PCIM-SAT can natively map arbitrary-order K-SAT problems and accelerate efficient multi/parallel-variable update heuristics. Leveraging SRAM-based mixed-signal in-memory computing and a new probabilistic SAT-solving algorithm, it achieves 100% solvability on 50-variable 3-SAT (satisfiable) instances in $5.5\mu s$, while consuming 265nJ (Fig. 10.10.6). The locally dense yet globally sparse structure of K-SAT problems, combined with the use of p-bits, enables seamless scalability to multi-tile architectures [15], where all graph computations can be performed locally within each tile, eliminating the need for analog reductions across tiles and limiting inter-tile communication to simple digital exchanges of variable and clause values.



Figure 10.10.1:

Basic concepts of K-SAT; introduction to the new multi-variable update SAT algorithm; challenges with prior SAT solver ASICs and design highlights of our solver architecture.



Figure 10.10.2:

Top-level architecture and workflow of proposed solver.



Figure 10.10.3:

Detailed schematic of the mixed-signal in-memory computing fabric and probabilistic bit.



Figure 10.10.4:

Measurement results of the probabilistic-bits; schematic of variable update logic and timing diagram of solver.



Figure 10.10.5:
Measurement results of PCIM-SAT on different SAT benchmarks and comparison with other solvers. Multi-thread mode implemented for comparison.



Figure 10.10.6:
Feature and performance comparison with prior SAT solver ASICs.



Figure 10.10.7:
Die micrograph and performance summary.

ACKNOWLEDGEMENT

This work is supported by the Defense Advanced Research Projects Agency (DARPA) under Air Force Research Laboratory (AFRL) contract no FA8650-23-3-7313. The authors thank Giacomo Pedretti, Thomas Van Vaerenbergh, Raymond Beausoleil, John Paul Strachan, Ignacio Rozada and other members of the DARPA QuICC team for valuable discussion and feedback.

IEEE Personal Account

CHANGE USERNAME/PASSWORD

Purchase Details

PAYMENT OPTIONS

VIEW PURCHASED DOCUMENTS

Profile Information

COMMUNICATIONS PREFERENCES

PROFESSION AND EDUCATION

TECHNICAL INTERESTS

Need Help?

US & CANADA: +1 800 678 4333

WORLDWIDE: +1 732 981 0060

CONTACT & SUPPORT

Follow

f

@

in

▶

X

About IEEE *Xplore* | Contact Us | Help | Accessibility | Terms of Use | Nondiscrimination Policy | IEEE Ethics Reporting  | Sitemap | IEEE Privacy Policy

A public charity, IEEE is the world's largest technical professional organization dedicated to advancing technology for the benefit of humanity.

© Copyright 2026 IEEE - All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies.